

Practical Group Steiner Tree Algorithms for Web Applications with Many Groups

Qicheng Shan
522022330011@smail.nju.edu.cn
Nanjing University
State Key Laboratory for Novel
Software Technology
Nanjing, Jiangsu, China

Yuxuan Yang
yxyang@smail.nju.edu.cn
Nanjing University
State Key Laboratory for Novel
Software Technology
Nanjing, Jiangsu, China

Gong Cheng*
gcheng@nju.edu.cn
Nanjing University
State Key Laboratory for Novel
Software Technology
Nanjing, Jiangsu, China

Abstract

The Group Steiner Tree Problem (GSTP) has been popularly used to formulate graph-based tasks on the Web where the number of groups (i.e., g), e.g., representing the number of keywords in knowledge graph search, is assumed to be small. This assumption never holds in emerging tasks such as the summarization of knowledge graphs, where g represents the number of distinct entity description patterns that increases with graph order (i.e., n) and reaches 10^5 in practice. Existing algorithms become impractical, since they are optimized for large n but not for large g . In this paper, we devise novel approximation algorithms for GSTP that exhibit scalability in relation to both n and g . When g is large, our algorithms outperform existing algorithms that have a comparable approximation ratio by orders of magnitude in running time, showing their unique ability to practically support such challenging Web applications.

CCS Concepts

• Mathematics of computing → Graph algorithms.

Keywords

Group Steiner Tree; Approximation Algorithm; Knowledge Graph

ACM Reference Format:

Qicheng Shan, Yuxuan Yang, and Gong Cheng. 2026. Practical Group Steiner Tree Algorithms for Web Applications with Many Groups. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792210>

1 Introduction

Various graph representations have been devised to model Web data such as social networks and knowledge graphs (KGs). A popular tool for formulating tasks on these graphs is the Group Steiner Tree Problem (GSTP) [7]. On a graph with n nodes and m weighted edges, given a query consisting of g groups each being a subset of nodes in the graph, GSTP aims to find a minimum-weight Group Steiner Tree (GST) that contains at least one node from each group. This NP-hard optimization problem has been applied in many scenarios.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792210>

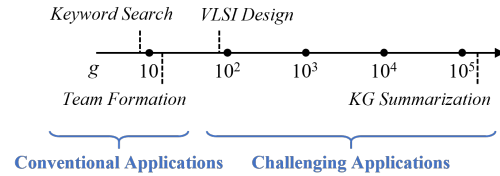


Figure 1: Maximum of g in real applications of GSTP.

Scenario 1: Keyword Search. In this classic task that has received renewed research interest, each keyword is matched with a group of entity nodes in a KG [11, 12, 16, 23] or nodes in a data graph [9, 15, 21], and important edges have small weights. A minimum-weight GST represents an optimal answer that shows the most important relationship between the relevant nodes.

Scenario 2: Team Formation. In a social network, each group of expert nodes share a skill, and edge weight represents communication cost. A minimum-weight GST represents a team of experts with the required skills at the minimum communication cost [8].

Scenario 3: KG Summarization. In this emerging task [17], a good summary of KG is a subgraph that compactly presents the various entity description patterns (EDPs) in the KG [19]. Each group of entity nodes have a common EDP, i.e., described with the same set of classes and properties. A minimum-weight GST, i.e., having the smallest number of unweighted edges, represents the most compact pattern-coverage summary [18], which has found application in KG profiling, search, and exploration [17].

Research Challenge. Conventional Web applications such as keyword search and team formation feature a small number of groups, as shown in Figure 1: $g \leq 10$ represents the number of search keywords [9, 11], and $g \leq 20$ represents the number of required skills [20]. Consequently, existing algorithms focus on scalability in relation to n (and m) rather than g . For example, among the practical GSTP algorithms, the time complexity of KeyKG⁺ [11] and PartialOPT [15] is cubic and exponential in g , respectively. This high complexity hinders their use in emerging Web applications such as KG summarization, where g represents the number of distinct EDPs and can be as large as n , exceeding 10^5 in practice [18, 19]. A similar case is in the application of very large-scale integration (VLSI) [22], where g represents the number of pins on a chip to be interconnected for routing and is around 10^2 . Empirical evidence shows that, on million-scale graphs, while KeyKG⁺ solves keyword search with $g \leq 10$ in milliseconds [11], it uses more than an hour to summarize KG with $g > 10^3$ [18] to an unreasonable long time

Algorithm 1: GSTGrow

Input: Graph $G = \langle V, E, w \rangle$; Query $Q = \{K_1, K_2, \dots, K_g\}$.
Output: GST $T = \langle V_T, E_T \rangle$.

```

1  $\langle U_r, r \rangle \leftarrow \text{CoreTerminals}(G, Q)$ 
2  $V_T \leftarrow \{r\}, E_T \leftarrow \emptyset$ 
3  $S \leftarrow \{r\}, X \leftarrow U_r \setminus \{r\}$ 
4 while  $X \neq \emptyset$  do
5    $x \leftarrow$  a core terminal in  $X$ 
6    $s \leftarrow \arg \min_{s' \in S} \text{dist}(s', x)$ 
7    $p \leftarrow$  a shortest path between  $s$  and  $x$ 
8    $p' \leftarrow \text{PathTrim}(p, \{x\}, V_T)$ 
9    $V_T \leftarrow V_T \cup (p' \cap V), E_T \leftarrow E_T \cup (p' \cap E)$ 
10   $S \leftarrow S \cup \{x\}, X \leftarrow X \setminus \{x\}$ 
11 return  $T$ 
```

for larger g . Therefore, the surge of g has become the new bottleneck in solving GSTP formulated in emerging Web applications. Novel algorithms that scale well to both g and n are needed, and such practical algorithms are expected to have a guaranteed worst-case approximation ratio that matches the guarantee of state-of-the-art practical algorithms including KeyKG⁺ [11] and ImprovAPP [15].

Our Work. We fill the gap with two novel GSTP algorithms: GSTGrow and GSTMerge. In GSTGrow, we grow a tree with the shortest paths between groups selected from a carefully restricted search space such that the time complexity is controlled to be linear in both g and n , while maintaining the same guaranteed approximation ratio as KeyKG⁺ and ImprovAPP. To cost-effectively expand the search space of GSTGrow so that the approximation ratio is empirically improved without significantly increasing the time complexity, we propose partition-based shortest path selection and incorporate it into GSTMerge, an algorithm in a tree-merging fashion. In summary, our contributions in this paper include the following:

- the first research attempt to solve GSTP instances with many groups to support emerging Web applications, and
- two practical approximation algorithms offering different trade-offs between time cost and result quality.

2 Problem Definition

Graph Notation. In a simple undirected weighted graph $G = \langle V, E, w \rangle$, V is a set of n nodes, $E \subseteq V \times V$ is a set of m edges, and $w : E \rightarrow \mathbb{R}^{0+}$ is a function assigning each edge a non-negative weight.¹ We refer to a path as a set of its constituent nodes and edges, so for a path p in $G = \langle V, E, w \rangle$, we retrieve its nodes and edges through $p \cap V$ and $p \cap E$, respectively. The length of p is the sum of the weights of its constituent edges: $\text{len}(p) = \sum_{e \in p \cap E} w(e)$. The distance $\text{dist}(u, v)$ between two nodes $u, v \in V$ is the length of a shortest path between them if one exists or infinity otherwise.

GSTP. A query is a set of g groups $Q = \{K_1, K_2, \dots, K_g\}$ that could overlap, where each group $K_i \subseteq V$ is a set of nodes called *terminals*. Given $G = \langle V, E, w \rangle$ and $Q = \{K_1, K_2, \dots, K_g\}$, a *Group Steiner Tree* (GST) $T = \langle V_T, E_T \rangle$ is a tree in G where $V_T \subseteq V, E_T \subseteq E$, and T connects all the groups, i.e., $V_T \cap K_i \neq \emptyset$ for all $K_i \in Q$.

¹Our algorithms are orthogonal to the concrete implementation of w . Our experiments will use standard weighting functions.

Algorithm 2: CoreTerminals

Input: Graph $G = \langle V, E, w \rangle$; Query $Q = \{K_1, K_2, \dots, K_g\}$.
Output: Set of core terminals U_r ; Core terminal r .

```

1 Swap  $K_1$  with the smallest  $K_i \in Q$ 
2 foreach  $v_1 \in K_1$  do
3    $U_{v_1} \leftarrow \{v_1\}$ 
4    $W_{v_1} \leftarrow 0$ 
5   for  $i \leftarrow 2$  to  $g$  do
6      $v_i \leftarrow \arg \min_{v \in K_i} \text{dist}(v, v_1)$ 
7      $U_{v_1} \leftarrow U_{v_1} \cup \{v_i\}$ 
8      $W_{v_1} \leftarrow W_{v_1} + \text{dist}(v_i, v_1)$ 
9  $r \leftarrow \arg \min_{v_1 \in K_1} W_{v_1}$ 
10 return  $\langle U_r, r \rangle$ 
```

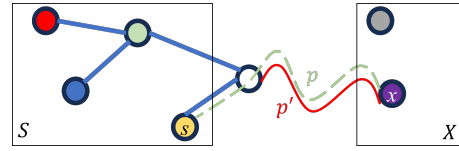


Figure 2: An illustration of PathTrim in GSTGrow. Colored nodes are core terminals.

With an abuse of notation, the weight of T is the sum of weights of its constituent edges: $w(T) = \sum_{e \in E_T} w(e)$. The *Group Steiner Tree Problem* (GSTP) aims to find a minimum-weight GST. GSTP is an NP-hard optimization problem and is also NP-hard to approximate within a factor of $O(\log^{2-\epsilon} g)$ [6]. In fact, to our knowledge, all known GSTP algorithms that have a practical running time on large graphs have an approximation ratio of at least $g - 1$, such as KeyKG⁺ [11] and ImprovAPP [15].

GSTP with Many Groups. In this paper, we focus on GSTP instances in which g is large, e.g., growing significantly with n , and thus cannot be regarded as a small constant in algorithm analysis.

3 GSTGrow: A Tree-Growing Algorithm

Our basic algorithm, GSTGrow, selects from each group a particular terminal called *core terminal* that has the smallest distance to the core terminal in the smallest group. A tree is repeatedly grown with a shortest path that connects to a core terminal not yet in the tree until the tree connects all the core terminals. Within this tree-growing paradigm, the novelty of GSTGrow is in finding shortest paths in a carefully restricted search space that employs distances only among terminals, which is distinguished from existing tree-growing algorithms such as KeyKG⁺ [11] and helps to fundamentally reduce the time complexity from cubic to linear in g .

3.1 Algorithm Design

GSTGrow. In Algorithm 1, we invoke CoreTerminals (detailed later) to select a set U_r of g core terminals, one from each group in the query Q , including a special $r \in U_r$ (line 1) which will be useful to analyze the approximation ratio. We initialize a tree $T = \langle V_T, E_T \rangle$ with r (line 2). Let S and X maintain the sets of core terminals inside and outside T , respectively (line 3). Repeatedly, we grow T until it

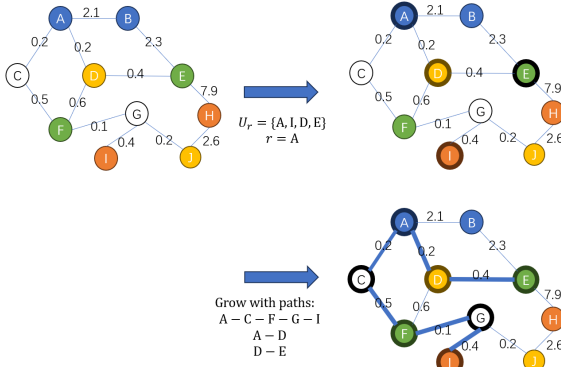


Figure 3: A running example of GSTGrow. Colored nodes are terminals, and each color represents a group.

contains all the core terminals (lines 4–10). In each iteration, we take a core terminal $x \in X$ (line 5), and find its nearest core terminal $s \in S$ (line 6), i.e., we use distances only among terminals. We find a shortest path p between s and x (line 7) and add it to T so that x will be in T (line 10). In fact, to ensure T is acyclic, we add not p but its minimal subpath p' that connects x from V_T (line 9), as illustrated in Figure 2, by invoking `PathTrim` which scans p starting from x until reaching the first vertex in V_T and ignores the remaining edges in p (line 8). When X becomes empty, i.e., T contains all the core terminals, T is a GST and returned (line 11).

CoreTerminals. In Algorithm 2, we enumerate all terminals in a group $K_1 \in Q$ (line 2), so for efficiency reasons, we choose the smallest group as K_1 (line 1). For each terminal $v_1 \in K_1$, we find its nearest terminal v_i in each group K_i (lines 5–6). These g terminals form a candidate set U_{v_1} of core terminals to be returned (lines 3 and 7). Let W_{v_1} be the sum of distances between v_1 and the other terminals in U_{v_1} (lines 4 and 8). Among all the $|K_1|$ candidate sets, U_r corresponding to the smallest sum of distances is returned (lines 9–10) because this set of core terminals are close to r , so they can be connected via r by short paths to form a small-weight GST.

Running Example. In Figure 3, assume $K_1 = \{A, B\} \in Q = \{\{A, B\}, \{H, I\}, \{D, J\}, \{E, F\}\}$. `CoreTerminals` selects $U_A = \{A, I, D, E\}$ instead of $U_B = \{B, I, D, E\}$, i.e., $r = A$, from which GSTGrow grows T with three paths $A - C - F - G - I$, $A - D$, and $D - E$, connecting to the other core terminals I , D , and E , respectively, forming a GST.

3.2 Algorithm Analysis

3.2.1 Preliminaries. Following [11], we use space-efficient *Hub Labeling* (HL) indexes [1] to query the exact distance and the shortest path. Here we briefly review the relevant concepts and results.

Static HL. Given a graph $G = \langle V, E, w \rangle$, a *static HL* [11] precomputes, for each node $v \in V$, its distance from each *hub* node and stores the result in its *label*, which is an ordered set of nodes $L_v \subseteq V$. For any two nodes $u, v \in V$, the existence of a shared hub $h \in L_u \cap L_v$ on a shortest path between u and v is guaranteed, so L_u and L_v can be linearly scanned to calculate $\text{dist}(u, v) = \min_{h \in L_u \cap L_v} \text{dist}(u, h) + \text{dist}(v, h)$ in $O(l)$ time where $l \in O(n)$

represents the size of the largest label. This *distance query* is denoted by $\text{DistQuery}(u, v)$. A static HL is constructed offline by executing a pruned Dijkstra's algorithm n times in $O(n(m + n \log n))$ time [1],² and its space complexity is $O(ln)$. By further storing the predecessor of each node, a shortest path between u and v can be restored in $O(n)$ time. This *shortest path query* is denoted by $\text{SPQuery}(u, v)$.

Dynamic HL. If the distances from a particular set of nodes $U \subseteq V$ will be frequently queried, their labels can be merged beforehand into a *dynamic HL* [11], which is an n -dimensional array that stores the smallest distance from U to each hub in U 's labels. A dynamic HL is constructed online from the static HL in $O(n + l|U|)$ time.³ Thanks to its random access in $O(1)$ time, the distance between U and any node $v \in V$, denoted by $\text{DistQuery}(U, v)$, can be queried in $O(l)$ time by linearly scanning L_v and directly accessing the array to obtain their intersection, which is faster than $|U|$ distance queries of a static HL in $O(l|U|)$ time. The shortest path, denoted by $\text{SPQuery}(U, v)$, can also be queried in $O(n)$ time.

3.2.2 Time Complexity and Approximation Ratio.

THEOREM 3.1. GSTGrow has a time complexity of $O(gln)$.

PROOF. The full proof is in Appendix A.1. The key idea is to use dynamic HLs for each $K_i \in \{K_2, \dots, K_g\}$ and for S . $\square$

The actual running time of GSTGrow, as we shall see in the experimental results, is much better than the upper bound $O(gln)$ because, generally, l is in $O(1)$ [13] and n here is used to limit the group size and the path hops that are also small in practice.

When n and g are both large, storing $g - 1$ array-based dynamic HLs for $K_2, \dots, K_g$ requires $\Theta(gn)$ memory which may be infeasible. Instead, dynamic HLs can be implemented with memory-efficient variable-length lists or replaced with a static HL, but the time complexity of GSTGrow will increase to $O(gn^2)$ or $O(gln^2)$, respectively.

THEOREM 3.2. GSTGrow has a worst-case approximation ratio of $g - 1$, i.e., the weight of T returned by GSTGrow is at most $g - 1$ times as large as that of a minimum-weight GST.

PROOF. The full proof is in Appendix A.2, based on the optimality of s in GSTGrow and of r and v_i in `CoreTerminals`. $\square$

This worst-case result is possible as illustrated in Appendix A.3, but tends to occur rarely in practice. The actual approximation ratio of GSTGrow, as we shall see in the experimental results, is very small and useful for downstream tasks.

3.2.3 Comparison with Existing Algorithms. Both using dynamic HLs, the time complexity $O(gln)$ of GSTGrow significantly outperforms `KeyKG+`'s $O(n^2g + ng^3)$ [11] when g is large, making GSTGrow more suitable for our problem setting. This gain is achieved without sacrificing the guaranteed approximation ratio; the two algorithms have the same worst-case approximation ratio of $g - 1$.

Despite the good empirical results, the guarantee of $g - 1$ may not be theoretically satisfactory when g is large. However, it aligns with the best theoretical result that the latest practical GSTP algorithms have achieved [11, 15]. All known GSTP algorithms that have a

²This offline time is not included in the running time of GSTP algorithms.

³Since U depends on the online query, this construction time is included in the running time of GSTP algorithms.

Algorithm 3: GSTMerge

Input: Graph $G = \langle V, E, w \rangle$; Query $Q = \{K_1, K_2, \dots, K_g\}$.
Output: GST $T = \langle V_T, E_T \rangle$.

```

1  $\langle U_r, r \rangle \leftarrow \text{CoreTerminals}(G, Q)$ 
2  $V_T \leftarrow U_r, E_T \leftarrow \emptyset$ 
3 foreach  $v \in V$  do
4    $\text{MakeSet}(v)$ 
5  $D \leftarrow U_r$ 
6 while  $|D| > 1$  do
7    $\langle D_S, X \rangle \leftarrow$  a bipartition of  $D$  such that  $r \in D_S$ ,
    $|D_S| = \lfloor \frac{\tau}{\tau+1} |D| \rfloor$ , and  $|X| = \lceil \frac{1}{\tau+1} |D| \rceil$ 
8    $S \leftarrow U_r \cap \bigcup_{v \in D_S} \text{Set}(\text{Find}(v))$ 
9   foreach  $\tilde{x} \in X$  do
10     $\langle s, x \rangle \leftarrow \arg \min_{\langle s', x' \rangle \in S \times (U_r \cap \text{Set}(\text{Find}(\tilde{x})))} \text{dist}(s', x')$ 
11     $p \leftarrow$  a shortest path between  $s$  and  $x$ 
12     $p' \leftarrow$ 
      PathTrim( $p, \text{Set}(\text{Find}(\tilde{x})), V_T \setminus \text{Set}(\text{Find}(\tilde{x}))$ )
13     $V_T \leftarrow V_T \cup (p' \cap V), E_T \leftarrow E_T \cup (p' \cap E)$ 
14    foreach  $(u, v) \in p' \cap E$  do
15       $\text{Union}(u, v)$ 
16     $D \leftarrow D \setminus \{\tilde{x}\}$ 
17 return  $T$ 
```

better guarantee of approximation ratio, as discussed in Section 6, are not scalable for large graphs.

4 GSTMerge: A Tree-Merging Algorithm

Recall that GSTGrow finds s based on x 's distance to S (line 6), which is a small set in early iterations and carries limited information for identifying an s that corresponds to a really short path p . To use more information, our improved algorithm GSTMerge initializes not a tree, but a forest of g trees—each consisting of a distinct core terminal—which are then repeatedly merged in pairs via a shortest path p until all core terminals are connected by a single tree. Compared with GSTGrow, the set of core terminals to be connected in each iteration of GSTMerge is intentionally partitioned in such a way that S consistently contains $\frac{\tau}{\tau+1}$ of these core terminals throughout all the iterations, including early iterations, where $\tau \geq 1$ is a parameter. This guaranteed large size of S offers rich information for identifying an s that corresponds to a short path p to improve the approximation ratio without significantly sacrificing time complexity.

4.1 Algorithm Design

For efficient tree merging, we implement a disjoint-set forest to maintain the node sets of trees. Standard operations include creating a new set with an element (MakeSet), merging two sets (Union), finding the representative of the set containing a given element (Find), and retrieving this entire set (Set).

GSTMerge. In Algorithm 3, after invoking CoreTerminals to select a set U_r of g core terminals (line 1), we initialize a forest $T = \langle V_T, E_T \rangle$ of g trees, each consisting of a distinct core terminal in U_r (lines 2–4). Let D maintain a maximum set of core terminals that are in different trees (line 5). Repeatedly, we merge trees until

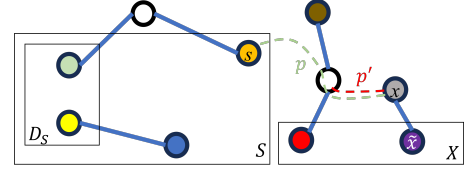


Figure 4: An illustration of PathTrim in GSTMerge. Colored nodes are core terminals.

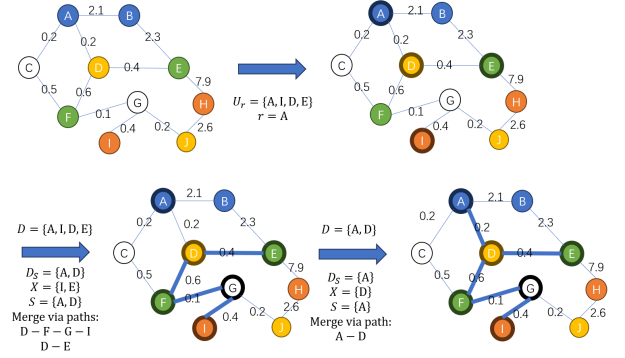


Figure 5: A running example of GSTMerge under $\tau = 1$. Colored nodes are terminals, and each color represents a group.

$|D| = 1$ (lines 6–16), i.e., all the core terminals are in a single tree. In each iteration, we partition D into two subsets, D_S and X , such that D_S contains $\frac{\tau}{\tau+1}$ of the core terminals in D , including r (line 7) which will be useful for analyzing the approximation ratio. Let S be the set of all core terminals in the trees containing D_S (line 8). Since S and X are in different trees, for each core terminal $\tilde{x} \in X$, we merge the tree containing $\tilde{x}$ with the nearest tree containing a core terminal in S (lines 9–16). Specifically, we find the nearest pair of core terminals $\langle s, x \rangle$ where $s \in S$ and x is from the tree containing $\tilde{x}$ (lines 9–10). We find a shortest path p between s and x (line 11) and add it to T to merge a pair of trees, so a larger τ will lead to a larger S and a shorter p , at the cost of more distance queries. To ensure T is acyclic, we invoke PathTrim to actually add a minimal subpath p' of p that connects the tree containing $\tilde{x}$ with any other tree (lines 12–15), as illustrated in Figure 4, and update D (line 16). When D becomes a singleton set, i.e., T is a tree containing all the core terminals, T is a GST and returned (line 17).

Running Example. In Figure 5, assume $K_1 = \{A, B\} \in Q = \{\{A, B\}, \{H, I\}, \{D, J\}, \{E, F\}\}$. CoreTerminals selects $U_A = \{A, I, D, E\}$ instead of $U_B = \{B, I, D, E\}$, i.e., $r = A$, from which GSTMerge initializes T with four trivial trees $\{A\}, \{I\}, \{D\}, \{E\}$. Assume $\tau = 1$. In the first iteration, GSTMerge bipartitions $D = \{A, I, D, E\}$ into $D_S = \{A, D\}$ and $X = \{I, E\}$, finds $S = \{A, D\}$, and merges two pairs of trees via two paths $D - F - G - I$ and $D - E$ into two trees $\{A\}$ and $\{D, E, F, G, I\}$, respectively. Compared with the running example of GSTGrow in Section 3.1 where $S = \{A\}$ in the first iteration, here $S = \{A, D\}$ is larger, based on which a shorter path is found to connect I with D than with A . In the second iteration, GSTMerge bipartitions $D = \{A, D\}$ into $D_S = \{A\}$ and $X = \{D\}$, finds $S = \{A\}$,

and merges a pair of trees via a path $A - D$, forming a GST, which has a smaller weight than that found by GSTGrow.

4.2 Algorithm Analysis

4.2.1 Time Complexity and Approximation Ratio.

THEOREM 4.1. *GSTMerge has a time complexity of $O(g \ln n + gn \log gn)$.*

PROOF. The full proof is in Appendix A.4. Key ideas include leveraging the fixed proportion $\frac{\tau}{\tau+1}$ to show that the while loop is executed $O(\log g)$ times, using dynamic HLs for each $K_i \in \{K_2, \dots, K_g\}$, for S , and for the set of core terminals in each tree, and using a disjoint-set forest with path compression for tree merging. $\square$

When n and g are both large, storing dynamic HLs in arrays may infeasibly require $\Theta(gn)$ memory. As an alternative solution, they can be stored in variable-length lists or replaced with a static HL, at the cost of increasing the time complexity of GSTMerge to $O(gn^2 + gn \log g)$ or $O(g \ln^2 n + g^2 l)$, respectively.

THEOREM 4.2. *GSTMerge has a worst-case approximation ratio of $g - 1$.*

PROOF. The full proof is in Appendix A.5, which basically resembles the proof of Theorem 3.2. $\square$

The actual running time and approximation ratio of GSTMerge, as we shall see in the experimental results, are much better than these upper bounds and are practical for downstream tasks.

4.2.2 Comparison with GSTGrow. Although GSTMerge guarantees the same approximation ratio as GSTGrow, in practice GSTMerge tends to find a smaller-weight GST thanks to the use of a larger set S for identifying s to find a shorter path p in early iterations. This improvement only insignificantly increases the time complexity from GSTGrow's $O(g \ln n)$ to GSTMerge's $O(g \ln n + gn \log gn)$, making GSTMerge *more cost-effective than* GSTGrow.

5 Experiments

We mainly experimented with GSTP instances from two challenging applications described in Section 1 where g is large: KG summarization (Section 5.2) and VLSI design (Section 5.3). We also experimented on a conventional application where g is small: keyword search, to assess the universality of our algorithms (Section 5.4).

Our code and data are open: <https://github.com/nju-websoft/GSTP-ManyGroups>.

5.1 Participating Algorithms

Our Algorithms. We implemented GSTGrow and GSTMerge with $\tau = 7$ by default. We will report the results of GSTMerge under different settings of this parameter in Section 5.5.

Baselines. We compared with three state-of-the-art GSTP algorithms that can scale to large graphs: KeyKG⁺ [11], ImprovAPP [15], and PartialOPT [15]. KeyKG⁺, ImprovAPP, and our algorithms have the same worst-case approximation ratio of $g - 1$. Whereas ImprovAPP is index-free, KeyKG⁺ uses static and dynamic HLs to accelerate distance and shortest path queries and is more comparable with our algorithms. PartialOPT is parameterized with $h \in [2, g]$ and has a worst-case approximation ratio of $g - h + 1$,

which outperforms the above algorithms when $h \geq 3$, but its time complexity is exponential in h . We followed [15] to set $h = 3$. We tried to increase h and observed no noticeable improvement in the approximation ratio but a sharp increase in running time.

In addition to PartialOPT, we are aware of other algorithms that have a worst-case approximation ratio better than $g - 1$, such as [2, 5]. However, to the best of our knowledge, they are mainly of theoretical interest and cannot scale to large graphs. These non-practical algorithms were not compared in our experiments.

5.2 Main Experiment 1: KG Summarization

5.2.1 Experimental Setup. This experiment was carried out on an Intel Xeon E3-1225 v6 CPU (3.30GHz) with 64GB of memory available for C++ programs. The setup of this experiment followed [18] where GSTP was used to formulate the task of KG summarization.

GSTP Instances. We followed Scenario 3 described in Section 1 to formulate KG summarization as GSTP on unweighted graph. From the dataset used in [18], we took all 11 challenging GSTP instances with $n \geq 100,000$ and $g \geq 5,000$ as listed in Appendix B.1.

Evaluation Metrics. We evaluated the efficiency of each algorithm by measuring its *running time* on each GSTP instance, allowing 24h for each run before marking it timeout. Following [11], the construction time for static HL was not included because it can be precomputed offline in real-world applications, but the construction time for dynamic HL was included because this query-dependent index can only be constructed online for each query.

We followed [18] to evaluate the effectiveness of each algorithm in KG summarization by measuring its *space saving* on each KG, i.e., the reduction in graph size relative to the original KG. A larger space saving represents a more compact and hence better summary of the KG derived from a GST of a smaller weight.

However, for the challenging GSTP instances in this experiment, it was computationally infeasible to employ any exact exponential-time algorithm to find the optimum solutions, so the actual approximation ratio of each algorithm was not accessible. We will analyze it in the next two experiments in Sections 5.3 and 5.4.

Implementation Details. Given the large value of g in this experiment, following our discussion in Sections 3.2.2 and 4.2.1, for CoreTerminals in our algorithms and its counterpart in KeyKG⁺ we could not afford to store $g - 1$ array-based dynamic HLs for $K_2, \dots, K_g$ in memory, but used a static HL, whose size and offline construction time were acceptable (see Appendix B.2). Other dynamic HLs used in the algorithms were implemented with arrays, except for those for the set of core terminals in each tree in GSTMerge implemented with variable-length lists.

5.2.2 Experimental Results. Table 1 reports the two metrics for each algorithm on each GSTP instance.

Comparison with Baselines. KeyKG⁺ encountered a timeout on most GSTP instances and used 2.36–16.27h to process the remaining instances due to its time complexity cubic in g . ImprovAPP encountered out-of-memory failures on all instances because it requested $\Theta(gn)$ memory to store shortest paths, which is infeasible when g and n are both large. Even if memory were sufficient, its running time would still be long due to its $g - 1$ executions of Dijkstra's algorithm for finding shortest paths. *These state-of-the-art algorithms, although practical for conventional applications of GSTP*

Table 1: Running Time and Space Saving in KG Summarization (best in bold, second best underlined)

KG	Running Time					Space Saving				
	KeyKG ⁺	ImprovAPP	PartialOPT	GSTGrow	GSTMerge	KeyKG ⁺	ImprovAPP	PartialOPT	GSTGrow	GSTMerge
#21	timeout	out-of-memory	11.83s	1.10s	<u>7.71s</u>	-	-	96.98%	<u>97.36%</u>	97.43%
#99	16.20h	out-of-memory	3.64s	0.84s	<u>1.09s</u>	99.17%	-	98.91%	99.13%	<u>99.16%</u>
#112	2.36h	out-of-memory	1.45s	0.05s	<u>0.11s</u>	96.60%	-	<u>96.56%</u>	96.53%	96.60%
#145	timeout	out-of-memory	3.67s	0.21s	<u>0.78s</u>	-	-	91.05%	<u>94.96%</u>	95.18%
#176	timeout	out-of-memory	6.11s	0.40s	<u>5.51s</u>	-	-	91.61%	<u>91.64%</u>	91.80%
#181	timeout	out-of-memory	3.35s	0.26s	<u>2.36s</u>	-	-	<u>89.47%</u>	89.38%	89.68%
#245	4.45h	out-of-memory	0.92s	0.05s	<u>0.11s</u>	95.64%	-	95.09%	95.44%	<u>95.58%</u>
#280	16.27h	out-of-memory	6.04s	0.14s	<u>0.36s</u>	98.54%	-	98.01%	98.46%	<u>98.51%</u>
#299	timeout	out-of-memory	19.20s	1.18s	<u>2.22s</u>	-	-	99.05%	<u>99.22%</u>	99.27%
#306	timeout	out-of-memory	31.83s	7.85s	<u>29.45s</u>	-	-	96.61%	<u>97.01%</u>	97.07%
#307	timeout	out-of-memory	7.97s	0.13s	<u>0.95s</u>	-	-	<u>96.07%</u>	<u>96.07%</u>	96.13%

with small values of g , became impractical for the emerging Web application of KG summarization, necessitating our research.

In contrast, our GSTGrow did not experience timeout or out-of-memory failure, and used at most 7.85s to process a GSTP instance, orders of magnitude faster than KeyKG⁺. Our algorithm exhibited efficiency for fundamentally resolving the performance bottleneck of the surge of g in solving GSTP. This was achieved only with a small loss of 0.04–0.20% in space saving relative to KeyKG⁺.

The time gap between GSTGrow and KeyKG⁺ was explained by their different numbers of distance queries executed over dynamic HLs as shown in Table 2: around $\frac{g^3}{2}$ in KeyKG⁺, versus g in GSTGrow, which demonstrates consistency with their time complexity.

In Table 1, the time gap between GSTGrow and PartialOPT was also large. GSTGrow was 4–61 times as fast as PartialOPT. Interestingly, despite its better guaranteed approximation ratio, PartialOPT's actual space saving was smaller than GSTGrow on most GSTP instances and smaller than GSTMerge on all the instances. It suggests that in terms of the actual approximation ratio in this application of GSTP, our algorithms are comparable with or better than one having a guaranteed sublinear approximation ratio.

Comparison between GSTGrow and GSTMerge. Compared to GSTGrow in Table 1, despite using relatively long but still acceptable time, GSTMerge consistently obtained larger space savings on all GSTP instances, increasing by up to 0.30% from GSTGrow. Therefore, GSTMerge arguably represents a better trade-off between time cost and result quality than GSTGrow for KG summarization.

5.3 Main Experiment 2: VLSI Design

We followed the description in Section 1 to formulate VLSI design as challenging GSTP instances with $g \geq 50$. For space reasons, we elaborate this experiment in Appendix B.3. Here we summarize the main conclusions: our GSTMerge was more cost-effective than KeyKG⁺, and exhibited a better trade-off between time cost and result quality than GSTGrow; ImprovAPP and PartialOPT showed limited ability, reflected by their frequent timeouts; the actual approximation ratios of our algorithms were much smaller than their proved worst-case result of $g - 1$, and rose slowly with g .

Table 2: Number of Distance Queries in KG Summarization

KG	Number of Distance Queries over Dynamic HLs		
	KeyKG ⁺	GSTGrow	GSTMerge
#21	-	69.6K	69.6K
#99	108.4B	6.0K	6.0K
#112	119.1B	6.2K	6.2K
#145	-	24.0K	24.0K
#176	-	65.1K	65.1K
#181	-	50.4K	50.4K
#245	85.9B	5.5K	5.5K
#280	680.8B	11.0K	11.0K
#299	-	23.2K	23.2K
#306	-	165.2K	165.2K
#307	-	19.6K	19.6K

5.4 Universality Study: Keyword Search

The GSTP instances used in this experiment come from keyword search on KGs, a conventional Web application where g is small. Although it does not match our problem setting of a large g , it represents a classic scenario for which existing algorithms were designed. Therefore, our goal here is to assess the universality of our algorithms in different Web scenarios.

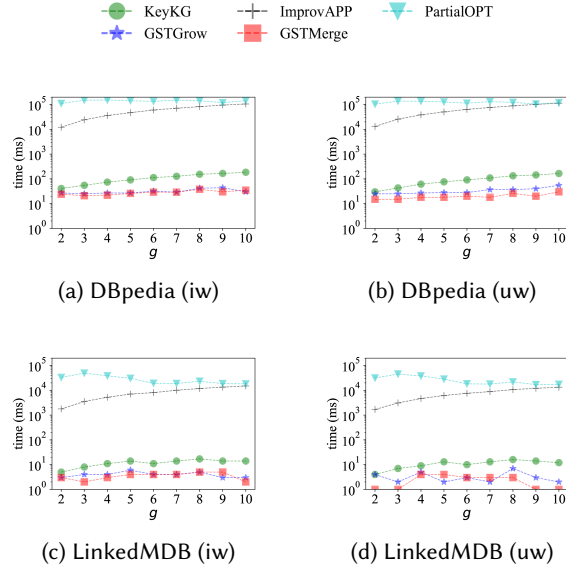
5.4.1 Experimental Setup. This experiment was carried out on an Intel Xeon E5-2643 v4 CPU (3.40GHz) with 629GB of memory available for C++ programs. The setup of this experiment followed [11].

GSTP Instances. We followed Scenario 1 described in Section 1 to formulate keyword search on KGs as GSTP on a weighted graph. From the dataset used in [11], we chose two largest KGs, DBpedia and LinkedMDB, where the edges are weighted in two different ways [23]: based on informativeness (denoted by iw), or uniformly weighted (denoted by uw). We reused keyword queries in [23], including 438 queries on DBpedia and 200 queries on LinkedMDB, where each query contains at most ten keywords, i.e., $g \leq 10$. We excluded trivial GSTP instances with $g = 1$, and excluded the instances for which no GST exists, e.g., due to graph disconnectivity.

Evaluation Metrics. We evaluated the efficiency of each algorithm by measuring its *running time* on each GSTP instance, allowing 200 seconds for each run before marking it timeout.

Table 3: Running Time and Approximation Ratio in Keyword Search (best in bold, second best underlined)

Graph	n	g	w	Running Time					Approximation Ratio				
				KeyKG ⁺	ImprovAPP	PartialOPT	GSTGrow	GSTMerge	KeyKG ⁺	ImprovAPP	PartialOPT	GSTGrow	GSTMerge
DBpedia	5,765,042	1–10	iw	74ms	35s	139s	<u>28ms</u>	24ms	<u>1.089</u>	1.061	1.093	1.150	1.137
			uw	60ms	38s	125s	<u>28ms</u>	17ms	<u>1.078</u>	<u>1.088</u>	1.090	1.118	1.110
LinkedMDB	748,593	1–10	iw	12ms	7s	25s	<u>5ms</u>	4ms	1.127	1.086	<u>1.125</u>	1.206	1.189
			uw	11ms	7s	24s	<u>4ms</u>	3ms	<u>1.145</u>	1.120	1.152	1.233	1.206

**Figure 6: Running time in keyword search.**

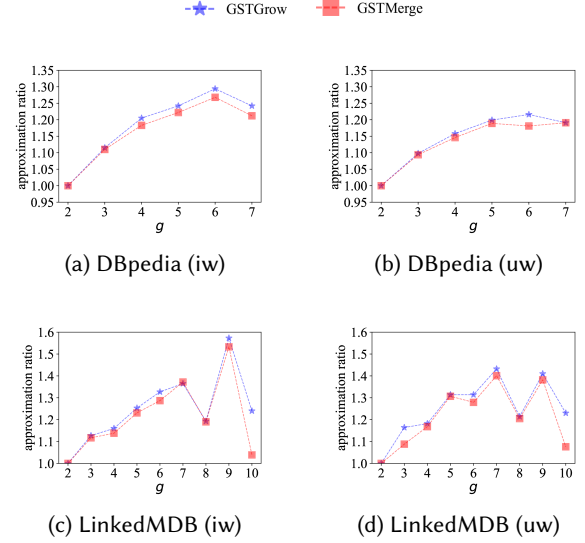
We followed [11] to evaluate the effectiveness of each algorithm in keyword search by measuring its actual *approximation ratio* on each GSTP instance, i.e., the ratio of the weight of GST found by the algorithm to that of an optimum GST. A smaller ratio represents a more important relationship between the relevant entities derived from a GST of a smaller weight. In this experiment, it was computationally feasible to employ an exact algorithm [9] to find the optimum solutions for most instances (except $g > 7$ on DBpedia), so the actual approximation ratio of each algorithm was accessible.

We averaged the running time over all queries on each KG in each edge weight setting, with timeout runs included as 200 seconds. We averaged the approximation ratio with timeout runs excluded.

Implementation Details. Given the small values of g in this experiment, for both our algorithms and KeyKG⁺ we implemented all dynamic HLs with arrays in memory.

5.4.2 Experimental Results. Table 3 reports the two metrics for each algorithm on each KG in each edge weight setting. Figures 6 and 7 report these metrics after dividing GSTP instances by g .

Comparison with Baselines. In Table 3, to solve an instance on DBpedia, ImprovAPP and PartialOPT used unsatisfactorily more than half a minute and two minutes, respectively, 3 orders of magnitude slower than our GSTGrow and GSTMerge despite achieving slightly better approximation ratios. KeyKG⁺ and our algorithms

**Figure 7: Approximation ratio in keyword search.**

used dynamic HLs and exhibited comparable performance. Our algorithms were 2–4 times as fast as KeyKG⁺, at the cost of raising the approximation ratio by 0.032–0.088. *Our algorithms, although not specifically designed for conventional GSTP applications with small values of g , are competitive with the state of the art, reflecting their potential as a universal solution for Web applications.*

In Figure 6, by increasing g , while the running time of KeyKG⁺ and ImprovAPP grew steadily, the performance of GSTGrow and GSTMerge was relatively stable. *It demonstrates the better scalability of our algorithms in relation to g .*

Actual Approximation Ratio. In Table 3, the approximation ratios of our GSTGrow and GSTMerge were very small, at most 1.233. Furthermore, in Figure 7, their approximation ratios fluctuated or rose slowly with g . *It suggests that, despite their guaranteed approximation ratio of $g-1$, the actual approximation ratios of our algorithms are consistently very small in this application of GSTP.*

5.5 Parameter Study

Our GSTMerge has a parameter $\tau \geq 1$. We analyzed its influence on the trade-off between the time cost and the result quality of GSTMerge. Based on consistent observations in different applications, we recommend choosing τ from 5–7 for a reasonable trade-off. The details are provided in Appendix B.4.

Table 4: Comparison with Existing GSTP Algorithms

Algorithm	Time Complexity	Approx. Ratio
DPBF	$O(3^g n + 2^g((g + \log n)n + m))$	1
PrunedDP++	$O(3^g n + 2^g((g + \log n)n + m))$	1
KeyKG ⁺	$O(n^2 g + n g^3)$	$g - 1$
ImprovAPP	$O(g(m + n \log n + n^2 + n \log g))$	$g - 1$
PartialOPT	$O(n(gn + 3^h n + 2^h(m + hn + n \log n)))$	$g - h + 1$
GSTGrow	$O(g \ln)$	$g - 1$
GSTMerge	$O(g \ln + g n \log g n)$	$g - 1$

6 Related Work

Despite its wide use in Web applications, GSTP is NP-hard and has an inapproximability of $O(\log^{2-\epsilon} g)$ [6]. Table 4 summarizes the guarantees of state-of-the-art exact and practical algorithms.

6.1 Practical Approximation Algorithms

To our knowledge, all existing practical algorithms that scale to large graphs guarantee an approximation ratio of around $g - 1$.

KeyKG⁺ [11] arguably represents the state of the art for its short running time in practice, thanks to its efficient tree-growing paradigm and the use of HL indexes to accelerate distance and shortest path queries, which are also featured in our GSTGrow. However, GSTGrow achieves a lower time complexity in relation to g and, in general, a better trade-off between time cost and result quality than KeyKG⁺, by growing a single tree instead of g trees to compare in KeyKG⁺, and growing with a shortest path selected based on the distance of a core terminal x to the set S of core terminals in the current tree T where $|S| \leq g$, as opposed to the counterpart of S in KeyKG⁺ comprising all the nodes in T where $|T|$ is usually much larger than g . In addition, our GSTMerge adopts a fundamentally different tree-merging paradigm and achieves an even better trade-off.

The more recent ImprovAPP [15] has the same worst-case approximation ratio as KeyKG⁺ and our algorithms, but in practice finds GSTs of smaller weights, since it takes more shortest paths into account. To obtain these paths, ImprovAPP executes Dijkstra's algorithm $g - 1$ times and stores the results in $\Theta(gn)$ memory. However, this time and space of ImprovAPP ceases to be practical when g and n are both large, as demonstrated by its consistent out-of-memory failures in KG summarization (Section 5.2) and its long running time inadequate to keyword search (Section 5.4).

PartialOPT [15], parameterized with an integer $h \in [2, g]$, combines two subtrees to form a GST: one connecting a terminal with $h - 1$ groups and the other connecting that terminal with the remaining $g - h$ groups. Its improved worst-case approximation ratio of $g - h + 1$ is achieved employing an exact algorithm [3] to find an optimum solution for the first subtree, at the cost of increasing the time complexity to be exponential in h . Consequently, PartialOPT is feasible only with a small h so that the improvement of result quality becomes negligible when g is large, as demonstrated by its consistently longer running time and larger approximation ratios than our GSTMerge in KG summarization (Section 5.2).

To conclude, existing practical GSTP algorithms have not guaranteed any approximation ratio that is significantly better than our algorithms. Although the current guarantee is arguably unsatisfactory from a theoretical point of view, our empirical results in

Web applications are encouraging, and our algorithms are—both theoretically and empirically—orders of magnitude faster when g is large. These features highlight the unique value of our contribution.

6.2 Exact and Non-Scalable Algorithms

For DPBF [3] and PrunedDP [9], the time complexity of these exact algorithms is exponential in g , limiting their use in Web applications where g is large, as demonstrated by their failure to compute optimum solutions in KG summarization (Section 5.2).

There are algorithms that guarantee an approximation ratio better than $g - 1$ [2, 5]. However, the polylogarithmic-approximation algorithm presented in [5] requires the construction of a separate graph for each group, not practical when g and n are large, and the 2-star algorithm [2] also has a very high time complexity in relation to g and n . Therefore, despite their better worst-case approximation ratios, these algorithms are mainly of theoretical interest and are not scalable for Web applications where n or g is large.

Another line of work [4, 14] transforms GSTP into a Steiner Tree Problem (STP), and their approximation ratios will be guaranteed only if an optimum STP solution is found. Since state-of-the-art exact STP solvers such as [10] rely on linear programming and require minutes to solve a small STP instance containing only tens of thousands of nodes, these transformation-based methods are not scalable for Web applications. We further clarify a key difference between these methods and our algorithms. Our GSTGrow and GSTMerge invoke CoreTerminals to obtain a set of core terminals U_r and then find a tree to connect U_r . This tree-finding step differs from a standard STP instance in that CoreTerminals specifies not only U_r but also a core terminal $r \in U_r$ that is used in a specific way to guarantee the approximation ratios of our algorithms.

7 Conclusion

We are among the first to address the surge of g in GSTP applications on the Web. Compared to state-of-the-art practical algorithms, our algorithms exhibit unparalleled scalability in relation to g , being orders of magnitude faster, and guarantee a matched approximation ratio. They will immediately benefit the emerging application of KG summarization where g is large, and are also suitable for serving as a universal tool to support a broad spectrum of GSTP applications on the Web such as keyword search where g is small.

In future work, a straightforward extension of our algorithms is to consider replacing HL indexes with other distance oracles [13] that are potentially more scalable. The current indexes we use [11] scale well to million-scale graphs, but their construction requires hours or days on larger graphs. Their study is beyond the scope of this paper but deserves investigation. As our long-term ambition, it would be invaluable to design a sublinear-approximation algorithm for GSTP that also scales well to both n and g .

Acknowledgments

This work was supported by the National Science and Technology Innovation 2030 Major Program (2025ZD0544900).

References

- [1] Takuya Akiba, Yoichi Iwata, and Yuichi Yoshida. 2013. Fast exact shortest-path distance queries on large networks by pruned landmark labeling. In *SIGMOD 2013*. 349–360. doi:10.1145/2463676.2465315
- [2] C. Douglass Bateman, Christopher S. Helvig, Gabriel Robins, and Alexander Zelikovskiy. 1997. Provably good routing tree construction with multi-port terminals. In *ISPD 1997*. 96–102. doi:10.1145/267665.267697
- [3] Bolin Ding, Jeffrey Xu Yu, Shan Wang, Lu Qin, Xiao Zhang, and Xuemin Lin. 2007. Finding Top-k Min-Cost Connected Trees in Databases. In *ICDE 2007*. 836–845. doi:10.1109/ICDE.2007.367929
- [4] C. W. Duin, A. Volgenant, and Stefan Voß. 2004. Solving group Steiner problems as Steiner problems. *Eur. J. Oper. Res.* 154, 1 (2004), 323–329. doi:10.1016/S0377-2217(02)00707-5
- [5] Naveen Garg, Goran Konjevod, and R. Ravi. 2000. A Polylogarithmic Approximation Algorithm for the Group Steiner Tree Problem. *J. Algorithms* 37, 1 (2000), 66–84. doi:10.1006/JAGM.2000.1096
- [6] Eran Halperin and Robert Krauthgamer. 2003. Polylogarithmic inapproximability. In *STOC 2003*. 585–594. doi:10.1145/780542.780628
- [7] Edmund Ihler. 1991. The Complexity of Approximating the Class Steiner Tree Problem. In *WG 1991*. 85–96. doi:10.1007/3-540-55121-2_8
- [8] Julio Juárez, Cipriano (Pano) Santos, and Carlos A. Brizuela. 2022. A Comprehensive Review and a Taxonomy Proposal of Team Formation Problems. *ACM Comput. Surv.* 54, 7 (2022), 153:1–153:33. doi:10.1145/3465399
- [9] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2016. Efficient and Progressive Group Steiner Tree Search. In *SIGMOD 2016*. 91–106. doi:10.1145/2882903.2915217
- [10] Daniel Rehfeldt and Thorsten Koch. 2023. Implications, conflicts, and reductions for Steiner trees. *Math. Program.* 197, 2 (2023), 903–966. doi:10.1007/S10107-021-01757-5
- [11] Yuxuan Shi, Gong Cheng, and Evgeny Kharlamov. 2020. Keyword Search over Knowledge Graphs via Static and Dynamic Hub Labelings. In *WWW 2020*. 235–245. doi:10.1145/3366423.3380110
- [12] Yuxuan Shi, Gong Cheng, Trung-Kien Tran, Evgeny Kharlamov, and Yulin Shen. 2021. Efficient Computation of Semantically Cohesive Subgraphs for Keyword-Based Knowledge Graph Exploration. In *WWW 2021*. 1410–1421. doi:10.1145/3442381.3449900
- [13] Christian Sommer. 2014. Shortest-path queries in static networks. *ACM Comput. Surv.* 46, 4 (2014), 45:1–45:31. doi:10.1145/2530531
- [14] Yahui Sun. 2018. Solving group Steiner problems as Steiner problems: the rigorous proof. *CoRR abs/1812.01678* (2018).
- [15] Yahui Sun, Xiaokui Xiao, Bin Cui, Saman K. Halgamuge, Theodoros Lappas, and Jun Luo. 2021. Finding Group Steiner Trees in Graphs with both Vertex and Edge Weights. *Proc. VLDB Endow.* 14, 7 (2021), 1137–1149. doi:10.14778/3450980.3450982
- [16] Xiaoping Wang and Gong Cheng. 2024. A Fast Hop-Biased Approximation Algorithm for the Quadratic Group Steiner Tree Problem. In *WWW 2024*. 312–321. doi:10.1145/3589334.3645325
- [17] Xiaxia Wang and Gong Cheng. 2024. A Survey on Extractive Knowledge Graph Summarization: Applications, Approaches, Evaluation, and Future Directions. In *IJCAI 2024*. 8290–8298.
- [18] Xiaxia Wang, Gong Cheng, Tengting Lin, Jing Xu, Jeff Z. Pan, Evgeny Kharlamov, and Yuzhong Qu. 2021. PCSG: Pattern-Coverage Snippet Generation for RDF Datasets. In *ISWC 2021*. 3–20. doi:10.1007/978-3-030-88361-4_1
- [19] Xiaxia Wang, Gong Cheng, Jeff Z. Pan, Evgeny Kharlamov, and Yuzhong Qu. 2023. BANDAR: Benchmarking Snippet Generation Algorithms for (RDF) Dataset Search. *IEEE Trans. Knowl. Data Eng.* 35, 2 (2023), 1227–1241. doi:10.1109/TKDE.2021.3095309
- [20] Xinyu Wang, Zhou Zhao, and Wilfred Ng. 2015. A Comparative Study of Team Formation in Social Networks. In *DASFAA 2015, Part I*. 389–404. doi:10.1007/978-3-319-18120-2_23
- [21] Qin Yuan, Ye Yuan, Zhenyu Wen, He Wang, Chen Chen, and Guoren Wang. 2022. Exploring Heterogeneous Data Lake based on Unified Canonical Graphs. In *SIGIR 2022*. 1834–1838. doi:10.1145/3477495.3531759
- [22] Martin Zachariasen and André Rohe. 2003. Rectilinear group Steiner trees and applications in VLSI design. *Math. Program.* 94, 2-3 (2003), 407–433. doi:10.1007/s10107-002-0326-x
- [23] Ke Zhang, Xiaoping Wang, and Gong Cheng. 2023. Efficient Approximation Algorithms for the Diameter-Bounded Max-Coverage Group Steiner Tree Problem. In *WWW 2023*. 199–209. doi:10.1145/3543507.3583257

A Full Proofs

A.1 Full Proof of Theorem 3.1

PROOF. The time complexity of *CoreTerminals* is dominated by distance queries (line 6). By constructing a dynamic HL for each $K_i \in \{K_2, \dots, K_g\}$ in $O(\ln)$ time, we find v_i by executing

DistQuery(K_i, v_1) in $O(l)$ time, so *CoreTerminals* has a time complexity of $O((g-1)\ln + |K_1|(g-1)l)$, or simply $O(g\ln)$.

In *GSTGrow*, besides invoking *CoreTerminals* (line 1), the while loop is executed $O(g)$ times (line 4), and its time complexity is dominated by distance queries and shortest path queries (lines 6–7). By constructing and maintaining a dynamic HL for S in $O(n + lg)$ time, we find s by executing *DistQuery*(S, x) in $O(l)$ time, and find p by executing *SPQuery*(s, x) in $O(n)$ time, so all while loops are completed in $O(n + lg + g(l + n))$ time, or simply $O(gn)$.

Overall, *GSTGrow* has a time complexity of $O(g\ln)$. $\square$

A.2 Full Proof of Theorem 3.2

PROOF. Let $T^* = \langle V_{T^*}, E_{T^*} \rangle$ be a GST of minimum weight.

For each path p' found in *GSTGrow*, we have

$$\text{len}(p') \leq \text{len}(p) = \min_{s' \in S} \text{dist}(s', x) \leq \text{dist}(r, x). \quad (1)$$

Since T is grown only with such p' , we have

$$w(T) \leq \sum_{x \in U_r \setminus \{r\}} \text{dist}(r, x) = W_r \leq W_{v_1^*}, \quad (2)$$

where $v_1^* \in V_{T^*} \cap K_1$ is a terminal in T^* . The last inequality is given by the way of choosing r in *CoreTerminals* (line 9).

According to the way of finding $U_{v_1^*}$ and $W_{v_1^*}$ in *CoreTerminals* (lines 3–8), we have

$$W_{v_1^*} = \sum_{v \in U_{v_1^*} \setminus \{v_1^*\}} \text{dist}(v, v_1^*) \leq \sum_{i=2}^g \text{dist}(v_i^*, v_1^*), \quad (3)$$

where each $v_i^* \in V_{T^*} \cap K_i$ is a terminal in T^* .

For each path p_i^* between v_i^* and v_1^* in T^* , we have

$$\text{dist}(v_i^*, v_1^*) \leq \text{len}(p_i^*) \leq w(T^*). \quad (4)$$

Combining Equations (2)(3)(4), we obtain

$$w(T) \leq W_{v_1^*} \leq \sum_{i=2}^g \text{dist}(v_i^*, v_1^*) \leq (g-1)w(T^*), \quad (5)$$

which completes the proof. $\square$

A.3 Worst-Case Approximation Ratio of *GSTGrow*

Figure 8 illustrates the worst-case approximation ratio of *GSTGrow*. In this example, the actual ratio is close to $g-1$ when ϵ is small.

A.4 Full Proof of Theorem 4.1

PROOF. For *CoreTerminals*, following the proof of Theorem 1, its time complexity is $O(g\ln)$ if using dynamic HLs.

In *GSTMerge*, besides invoking *CoreTerminals* (line 1), each while loop reduces the size of D to $\lfloor \frac{\tau}{\tau+1} |D| \rfloor$ (lines 6–7, 9, 16). Therefore, using k to denote the total number of times the while loop is executed, we have

$$g\left(\frac{\tau}{\tau+1}\right)^k \geq 1, \quad \text{i.e., } k \leq \frac{\ln g}{\ln(\tau+1) - \ln \tau}. \quad (6)$$

Since $\tau \geq 1$ is a constant, we have $k \in O(\log g)$.

The time complexity of the while loop is partially dominated by distance and shortest path queries (lines 10–11). In each loop, we construct a dynamic HL for S in $O(n + lg)$ time. We also construct g dynamic HLs for the sets of core terminals in g trees (e.g., $U_r \cap$

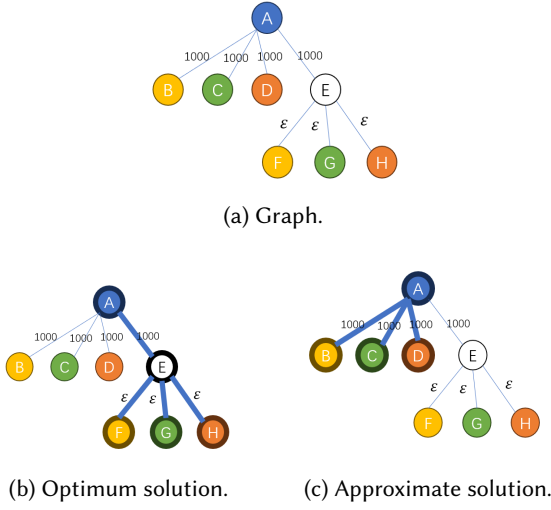


Figure 8: An example of the worst-case approximation ratio of GSTGrow. Colored nodes are terminals, and each color represents a group.

Set(Find($\tilde{x}$))) in the first loop in $O(g(n + l))$ time, and maintain them in the remaining loops in $O(gn)$ time. With dynamic HLs, we find $\langle s, x \rangle$ by at most n array accesses in $O(n)$ time, and find p by executing SPQuery(s, x) in $O(n)$ time. In summary, the time complexity for dynamic HLs throughout $k \in O(\log g)$ while loops is $O((n + lg) \log g + g(n + l) + gn + gn)$, or simply $O(gn \log g)$.

Regarding disjoint-set, throughout GSTMerge, there are n MakeSet operations (line 4) and at most $n - 1$ Union operations (line 15). Each while loop explicitly performs one Find operation for each node in D (lines 8, 10/12), so throughout k while loops the total number of times of explicitly performing Find is at most

$$\sum_{i=0}^{k-1} g\left(\frac{\tau}{\tau+1}\right)^i \leq g(\tau+1) \in O(g). \quad (7)$$

Moreover, each PathTrim implicitly performs at most n Find operations, one for each node in p (line 12). In summary, by implementing a disjoint-set forest with path compression, throughout GSTMerge there are n MakeSet operations, at most $n - 1$ Union operations, and $O(gn)$ Find operations. Their total time complexity is $O(gn \log n)$. In addition, each Find operation is followed by a Set operation (lines 8, 10/12). In each while loop, Set operations retrieve each node at most once, so throughout the k while loops the total time complexity of Set operations is $O(n \log g)$.

Overall, GSTMerge's time complexity is $O(gln + gn \log gn)$. $\square$

A.5 Full Proof of Theorem 4.2

PROOF. For each path p' found in GSTMerge, we have

$$\begin{aligned} \text{len}(p') \leq \text{len}(p) &= \min_{\langle s', x' \rangle \in S \times (U_r \cap \text{Set}(\text{Find}(\tilde{x})))} \text{dist}(s', x') \\ &\leq \text{dist}(r, \tilde{x}). \end{aligned} \quad (8)$$

The last inequality is given by the construction of D_S and S in GSTMerge (lines 7–8) which ensures $r \in D_S \subseteq S$.

Since T is grown only with such p' , and each p' is associated with a distinct $\tilde{x} \in U_r \setminus \{r\}$, we have

$$w(T) \leq \sum_{\tilde{x} \in U_r \setminus \{r\}} \text{dist}(r, \tilde{x}) = W_r. \quad (9)$$

The rest of the proof is omitted as it trivially resembles our proof of GSTGrow's approximation ratio in Appendix A.2. $\square$

B Additional Experiments

B.1 GSTP Instances in KG Summarization

Table 5 shows the KG and the size of the input of each GSTP instance used in the experiment of KG summarization.

Table 5: GSTP Instances in KG Summarization

ID	KG Name	n	g
#21	GeoSpecies Knowledge Base	1,768,818	69,677
#99	WordNet 2.0 (W3C)	1,037,838	6,008
#112	A linked dataset of Estonian organizations	192,406	6,200
#145	SIMPLE	457,154	24,027
#176	National Digital Library	758,333	65,184
#181	National Digital Library Works in the Public Domain	465,384	50,419
#245	The Mondial Database	111,545	5,561
#280	Hungarian WordNet RDF	732,285	11,084
#299	Encabezamientos de Materia de la BNE en SKOS	3,116,301	23,279
#306	European Nature Information System	4,533,712	165,239
#307	RISM Authority data	679,609	19,648

B.2 Static HLs in KG Summarization

Table 6 shows the size and construction time of each static HL used in the experiment of KG summarization.

Table 6: Static HLs in KG Summarization

KG	Size of Static HL	Construction Time
#21	2.4GB	3m29s
#99	4.2GB	6m50s
#112	49.3MB	4s
#145	205.7MB	16s
#176	262.4MB	20s
#181	179.3MB	14s
#245	124.5MB	10s
#280	122.3MB	9s
#299	1.9GB	3m33s
#306	14.6GB	34m57s
#307	66.1MB	6s

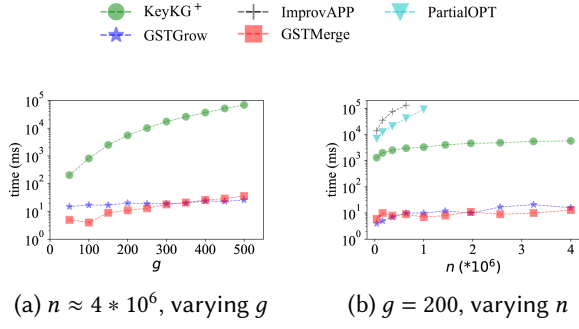


Figure 9: Running time in VLSI design.

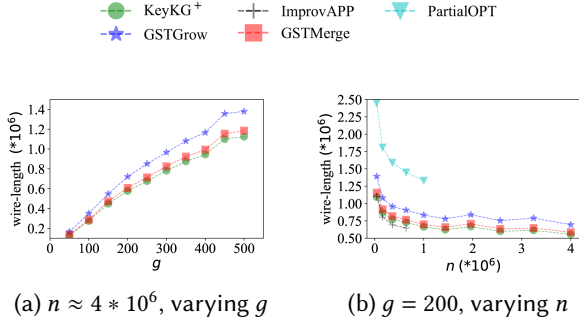


Figure 10: Wire-length in VLSI design.

B.3 Main Experiment 2: VLSI Design

B.3.1 Experimental Setup. This experiment was carried out on an Intel Xeon E5-2643 v4 CPU (3.40GHz) with 629GB of memory available for C++ programs. The setup of this experiment followed [22] where GSTP was used to formulate the task of VLSI design.

GSTP Instances. We followed the description in Section 1 and [22] to formulate VLSI design as GSTP. We synthesized GSTP instances with varying values of g and n on Hanan grid graphs, where the edges are weighted by rectilinear distances between plane points representing nodes. To evaluate the scalability of the algorithms in relation to g , we firstly fixed $n \approx 4 \times 10^6$ and for each $g \in \{50, 100, 150, \dots, 500\}$ we generated twenty instances. Similarly, to evaluate scalability in relation to n , we fixed $g = 200$ and generated twenty instances for each $n \in \{200^2, 400^2, 600^2, \dots, 2000^2\}$.

Evaluation Metrics. We evaluated the efficiency of each algorithm by measuring its *running time* on each GSTP instance, allowing 200 seconds for each run before marking it timeout.

We followed [22] to evaluate the effectiveness of each algorithm in VLSI design by measuring its *wire-length* on each chip. A smaller wire-length represents a smaller routing cost and hence a better VLSI design derived from a GST of a smaller weight.

We averaged the running time and wire-length over all 20 instances in each setting of g and n , with timeout runs excluded.

Implementation Details. In this experiment, HLs were not needed. The rectilinear distance was computed in constant time, and the shortest paths were easily obtained along the grid.

B.3.2 Experimental Results. Figures 9 and 10 report the two metrics for each algorithm.

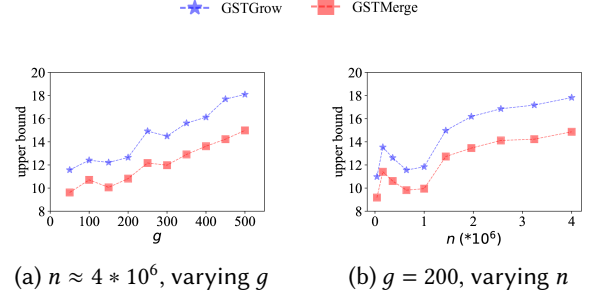


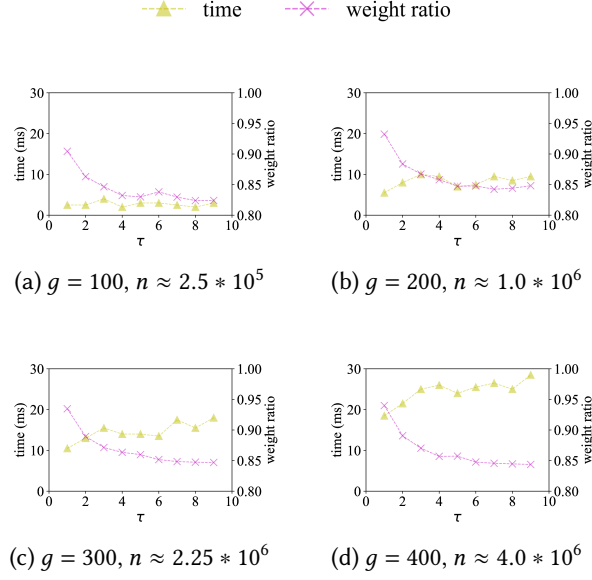
Figure 11: Upper bound on approximation ratio in VLSI design.

Comparison with Baselines. As shown in Figure 9, KeyKG⁺ was 1–3 orders of magnitude slower than our GSTGrow and GSTMerge. Although they scaled equally well to n in Figure 9(b), with a rising g in Figure 9(a), the time gaps between KeyKG⁺ and our algorithms became increasingly pronounced. *It demonstrates the better scalability of our algorithms in relation to g than KeyKG⁺.* This was achieved at a marginal cost of lifting wire-length as shown in Figure 10 where KeyKG⁺ and GSTMerge were close. *Therefore, GSTMerge is a generally more cost-effective algorithm than KeyKG⁺ for VLSI design.*

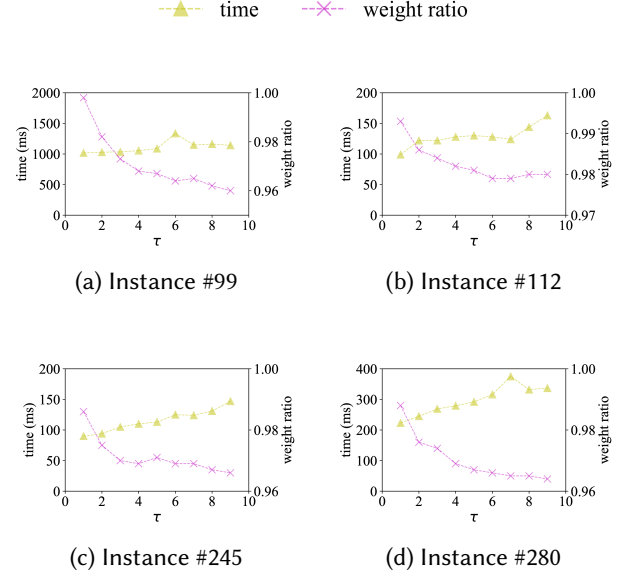
In Figure 9(b), ImprovAPP and PartialOPT were at least 3 orders of magnitude slower than our GSTGrow and GSTMerge when n was small, and quickly encountered timeout after increasing n . With a fixed $n \approx 4 \times 10^6$ in Figure 9(a), they both encountered a timeout even for the smallest value of $g = 50$. *It reflects the limited capability of these algorithms for supporting VLSI design.* On relatively small graphs where some ImprovAPP runs were completed successfully, its wire-length outperformed GSTMerge, as shown in Figure 10(b). It suggests that there is room to improve the approximation ratios of our algorithms for this application of GSTP.

Comparison between GSTGrow and GSTMerge. The two algorithms had a comparable running time in Figure 9, but their wire-lengths differed by about 20% in Figure 10. Our in-depth analysis revealed that the paths found in early iterations in GSTGrow were often very long due to the limited information it could use, which was exactly enriched in GSTMerge to find shorter routes. *Therefore, GSTMerge exhibited a better trade-off between time cost and result quality than GSTGrow for VLSI design.*

Actual Approximation Ratio. Although the optimum solutions for the challenging GSTP instances in this experiment could not be computed in reasonable time, we estimated a lower bound on the weight of an optimum solution as follows. For each GSTP instance, we greedily selected 5 groups that were farthest from each other, and employed an exact GSTP algorithm [9] to find an optimum solution to this GSTP sub-instance with $g = 5$. This was computationally feasible since g was small. The weight of this optimum solution to a sub-instance became a natural lower bound on the weight of an optimum solution to the original instance, based on which we measured an *upper bound on the approximation ratio* of our algorithms. This bound may be loose, but useful for observing whether the approximation ratio is close to the worst case of $g - 1$.

Figure 13: Effects of τ on GSTMerge in VLSI design.

In Figure 11, the upper bounds on the actual approximation ratios of our GSTGrow and GSTMerge were at most 18 and 15, respectively, much smaller than $g = 500$. Note that the actual approximation ratios could be much better than these bounds. *It suggests that, despite their proved worst-case approximation ratio of $g - 1$, the actual approximation ratios of our algorithms are much smaller in this application of GSTP.* More importantly, the bounds rose slowly when increasing g in Figure 11(a) and increasing n in Figure 11(b).

Figure 12: Effects of τ on GSTMerge in KG summarization.

B.4 Parameter Study

Our GSTMerge has a parameter $\tau \geq 1$. With a larger τ , the distances to more core terminals will be exploited to help find shorter paths and construct a GST of smaller weight, at the cost of executing more distance queries. We analyzed the influence of this parameter on the time cost and result quality of GSTMerge.

B.4.1 Experimental Setup. We reran our main experiments in Section 5.2 and Section 5.3, with GSTMerge configured to use different values of τ varying from 1 to 9.

Evaluation Metrics. Besides *running time*, to evaluate the effects of different values of τ on the relative quality of the result of GSTMerge, we measured *weight ratio*, i.e., the ratio of the weight of GST found by GSTMerge to that of GSTGrow as a baseline. A smaller ratio represents a better GST.

B.4.2 Experimental Results. Figure 12 reports the two metrics for GSTMerge using each value of τ on four representative GSTP instances in KG summarization. Figure 13 reports the average metrics for GSTMerge using each value of τ over all the instances in four representative settings of g and n in VLSI design. Our observations on other instances were similar and hence omitted.

In general, increasing τ from 1 to 5, the weight ratio of GSTMerge dropped rapidly in both applications, i.e., better GSTs were found after further considering a few more distances. However, continuing to increase τ to 7 or 9, the weight ratio converged or improved slowly. Meanwhile, the running time of GSTMerge increased at a generally steady rate with τ . Based on these consistent observations across different applications, we recommend choosing τ from 5–7 for a reasonable trade-off between time cost and result quality.